

The mathematics behind the bCCNN: the ccODP model, the network and its chain-ladder start

Explainer for `fit_odp_glm()` in `R/fit ODP GLM`, `bccnn_model()` and `fit_bccnn()` in `R/nn_models.R`, `R/loss functions.R`, `R/metrics.R` and the script `analysis/03_bCCNN/bCCNN fit.R`

Prepared for Francis

6 October 2026

Contents

Notation used in this document

1 What this document covers

2 Set-up: the triangle

3 The ccODP model

3.1 Assumptions

3.2 Identification

3.3 The Poisson deviance

3.4 The maximum likelihood equations

3.5 How `glm()` solves them: IWLS

3.6 The ccODP is the chain ladder

3.7 Dispersion

3.8 The unit of the payments

3.9 Periods without payments

4 The bCCNN network

4.1 Embedding layers

4.2 The skip connection

4.3 The feed-forward part

4.4 The output neuron and eq. (13)

4.5 Dropout

4.6 The initialisation: Paper C's eq. (14)

5 The loss as Keras sees it

5.1 Keras's "poisson" loss and the deviance

5.2 What is recorded after every step

6 Gradients

6.1 Derivative in the output

6.2 Output layer

6.3 Hidden layers (back-propagation)

6.4 Property 2: the gradient at the start

6.5 Property 3: the size of the first steps

7 From the fitted means to the reserves

8 Dispersion of the bCCNN

9 Diagnostics computed from the means

10 Where each formula is in the code

11 Numerical check of the formulas

References

Notation used in this document

Symbol	Meaning	Section
n	number of accident periods = number of development periods; here $n = 20$ (annual)	2
i, j	accident period $1, \dots, n$ and development period $1, \dots, n$ (1-based, as in the code; Paper C's development year j is $j - 1$ here)	2
$k = i + j - 1$	calendar period of cell (i, j) ; the valuation date is calendar period n	2
$\mathcal{O}, \mathcal{F}$	observed cells $\{i + j \leq n + 1\}$ (210 of them) and future cells $\{i + j > n + 1\}$ (190)	2
$Y_{i,j}, y_{i,j}$	incremental payments of cell (i, j) as a random variable and as observed, in units of u	2
u	unit of the payments in the fit, $u = 10^6$ (data: scale)	3.8
c, α_i, β_j	intercept, accident-period effect and development-period effect of the ccODP model, $\alpha_1 = \beta_1 = 0$	3
$\hat{c}, \hat{\alpha}_i, \hat{\beta}_j$	their maximum likelihood estimates	3
$\mu_{i,j}$	mean of cell (i, j) under the model at hand; $\hat{\mu}_{i,j}^{\text{cc}}$ for the fitted ccODP	3, 4
ϕ	dispersion: $\text{Var}(Y_{i,j}) = \phi \mu_{i,j}$	3
$D(\mathbf{y}, \boldsymbol{\mu}; \mathcal{S})$	Poisson deviance of the means $\boldsymbol{\mu}$ on the cells $\mathcal{S}$	3.3
$n_{\text{obs}}, n_{\text{par}}$	number of fitted cells and of ccODP parameters, $n_{\text{par}} = 2n - 1 = 39$	3.2
f_j	chain-ladder factor from development period j to $j + 1$	3.6
q_1, q_2, q_3	neurons of the three hidden layers, (20, 15, 10)	4
$W^{(h)}, b^{(h)}$	weights ($q_{h-1} \times q_h$) and biases of hidden layer h , $q_0 = 2$	4
$z^{(h)}(i, j)$	activations of hidden layer h for cell (i, j) ; $z^{(0)} = (\alpha_i, \beta_j)^\top$	4
p	dropout rate, $p = 0.1$; $m^{(h)}$ a dropout mask	4.5
w, c, B	weight of the skip connection, intercept and weights of the output neuron	4.4

Symbol	Meaning	Section
θ	all trainable parameters, $\theta = (W^{(1)}, b^{(1)}, \dots, B, w, c)$; 547 numbers	4
θ_0	the start of gradient descent, eq. (14) of Paper C	4.6
$a_{i,j}$	output before the exponential, $a = \log \mu$	4.4
$\ell(\theta)$	the loss Keras minimises (mean Poisson loss over the training cells)	5
N	number of training cells, $N = 210$ on the full triangle	5
$\delta_{i,j}$	derivative of the loss in the output $a_{i,j}$	6
η, ρ, ϵ	RMSprop learning rate, decay and stabiliser: 0.001, 0.9, 10^{-7}	6.5
$\hat{R}_i, \hat{R}$	reserve (predicted outstanding payments) of accident period i and in total	7
$g(i, j)$	cumulative development factor of accident period i to development period j	9

Abbreviations: ccODP, cross-classified over-dispersed Poisson; bCCNN, blended cross-classified neural network; CL, chain ladder; MLE, maximum likelihood estimate; IWLS, iteratively weighted least squares. “Paper C” is Gabrielli, Richman and Wüthrich (2020), *Neural network embedding of the over-dispersed Poisson reserving model*, Scandinavian Actuarial Journal 2020(1):1-29; its equation numbers are quoted as your code comments quote them ((3), (4), (5), (13), (14), Listing 4). “Härkönen” is Härkönen (2021), *On claims reserving with machine learning techniques*, Master thesis, Stockholm University, the thesis you supplied; its Section 2.3.3 restates Paper C and is cited by its own equation numbers where it is used.

1 What this document covers

This explainer writes out, formula by formula, what the bCCNN code computes on one 20×20 annual triangle: the cross-classified over-dispersed Poisson model that the chain ladder rests on, the neural network that is wrapped around it, the loss as Keras sees it, the gradients, and the properties of the starting point of gradient descent. The fitting procedure (which cells train, which cells stop the training, how the final network is chosen) is in the companion *The fitting procedure and training of the bCCNN*; the time-aware split is in *The time-aware split for the bCCNN*; interpretation, evaluation and uncertainty are in *The bCCNN as a whole*. The four documents share the notation above.

Three things in the code are not in Paper C and are marked where they appear: the rolling-origin early stopping (companion documents), the fact that the hidden layers use Keras’s default initialiser (Section 4.6, your `INFERRED` comment), and the handling of accident or development periods without payments (Section 3.9), which is in neither Paper C nor Härkönen.

How the maths was checked. Every formula of Sections 3 to 9 was recomputed in plain numpy, without Keras, on a synthetic 20×20 triangle with the dimensions of your data (210 observed cells, payments in

millions), and the two implementations were compared. The results are in Section 11. In short: the IWLS fit of Section 3 reproduces the chain-ladder reserve to a relative error of 5×10^{-15} and satisfies the marginal-total equations to 10^{-11} ; the network at its starting point reproduces the ccODP means to 10^{-14} ; the analytic gradients of Section 6 agree with central finite differences to a relative error of 10^{-7} or better in every parameter group; the parameter count agrees with Keras's 547; and the predicted behaviour of the first RMSprop step (Section 6.5) is observed exactly.

2 Set-up: the triangle

The portfolio has $n = 20$ accident periods (years) $i = 1, \dots, n$ and n development periods $j = 1, \dots, n$. A cell (i, j) holds the incremental payments $Y_{i,j}$ made in development period j on claims of accident period i , that is, in calendar period

$$k = i + j - 1. \quad (1)$$

At the valuation date, calendar period n , the observed cells are those with $k \leq n$ and the future cells those with $k > n$:

$$\begin{aligned} \mathcal{O} &= \{(i, j) : i + j \leq n + 1\}, & \mathcal{F} &= \{(i, j) : i + j > n + 1\}, \\ |\mathcal{O}| &= \frac{n(n+1)}{2} = 210, & |\mathcal{F}| &= 190. \end{aligned} \quad (2)$$

The quantity to predict is the sum of the future cells, the outstanding payments to development period n ,

$$R = \sum_{(i,j) \in \mathcal{F}} Y_{i,j}, \quad R_i = \sum_{j > n+1-i} Y_{i,j}. \quad (3)$$

In the simulated data R is known: `triangle_sets()` returns it as the matrix `test` (the lower triangle, `NA` above the diagonal). Payments after development period n exist in the simulation but are in no triangle and no model; they are returned apart as `tail_o` and reported on their own row of the results table. (The simulator's `triangle.csv` folds them into column n , which for accident period 1 is an observed cell; the script checks that folding and then uses `triangle_sets()`'s triangles, which stop at development period n .)

In the code the square `sets$full` holds all $Y_{i,j}$, `sets$upper` holds $\mathcal{O}$ (future cells `NA`), and every amount is divided by $u = 10^6$ before fitting (`dat_cfg$scale`), so the fit runs in millions; Section 3.8 shows this changes nothing but the intercept.

3 The ccODP model

3.1 Assumptions

Paper C, Section 2, and Härkönen, Section 2.1, eq. (1)-(3). The incremental payments of the observed cells are independent, with

$$\mathbb{E}[Y_{i,j}] = \mu_{i,j} = \exp\{c + \alpha_i + \beta_j\}, \quad \text{Var}(Y_{i,j}) = \phi \mu_{i,j}, \quad \phi > 0. \quad (4)$$

The model is called cross-classified because the two factors, accident period and development period, enter additively on the log scale: every accident period has the same development pattern $(e^{\beta_j})_j$, scaled by its

own level $e^{c+\alpha_i}$. No interaction between i and j is allowed; that is exactly what the network of Section 4 adds.

“Over-dispersed Poisson” means the variance is ϕ times the mean instead of equal to it. A concrete distribution with these moments is the scaled Poisson $Y = \phi P$ with $P \sim \text{Poisson}(\mu/\phi)$: then $\mathbb{E}[Y] = \phi \cdot \mu/\phi = \mu$ and $\text{Var}(Y) = \phi^2 \cdot \mu/\phi = \phi\mu$. The fit, however, only uses the first two moments: it is a quasi-likelihood fit (`family = quasipoisson(link = "log")`), and ϕ is estimated afterwards (Section 3.7). The payments must be non-negative, and `fit_odp_glm()` stops if an observed increment is negative (Härkönen replaces negative aggregates by zero; your simulated triangle has none).

3.2 Identification

The linear predictor $c + \alpha_i + \beta_j$ has $1 + n + n$ coefficients but only $2n - 1$ of them are identifiable: adding a constant to all α_i and subtracting it from c changes nothing, and likewise for the β_j . Paper C’s eq. (3) fixes

$$\alpha_1 = \beta_1 = 0, \quad (5)$$

so that c is the log mean of cell $(1, 1)$, α_i is the log ratio of accident period i to accident period 1 at the same development period, and β_j the log ratio of development period j to development period 1 within an accident period. In `fit_odp_glm()` this is R’s treatment contrasts for `y ~ origin + dev` with `origin` and `dev` as factors whose first level is period 1: the intercept is c , the coefficient `origini` is α_i and `devj` is β_j . The number of parameters is

$$n_{\text{par}} = 1 + (n - 1) + (n - 1) = 2n - 1 = 39, \quad (6)$$

against $n_{\text{obs}} = |\mathcal{O}| = 210$ fitted cells on the full triangle.

3.3 The Poisson deviance

For a mean vector μ and a set of cells $\mathcal{S}$, the (unscaled) Poisson deviance is Paper C’s eq. (4):

$$D(\mathbf{y}, \mu; \mathcal{S}) = 2 \sum_{(i,j) \in \mathcal{S}} \left(\mu_{i,j} - y_{i,j} + y_{i,j} \log \frac{y_{i,j}}{\mu_{i,j}} \right), \quad 0 \log 0 := 0. \quad (7)$$

It is `poisson_deviance(y, mu)` in `R/loss_functions.R`, which drops the cells where `y` is NA (that is how the same function scores the observed triangle, a validation mask or the true lower triangle). Each summand is non-negative and zero only when $\mu_{i,j} = y_{i,j}$, because $x \mapsto \mu - x + x \log(x/\mu)$ is the Bregman divergence of $x \log x$.

Where it comes from. The Poisson log-likelihood of the observed cells, with the means scaled by ϕ as in the scaled-Poisson representation, is up to a constant

$$\ell(\mu) = \frac{1}{\phi} \sum_{\mathcal{S}} (y_{i,j} \log \mu_{i,j} - \mu_{i,j}). \quad (8)$$

The saturated model sets $\mu_{i,j} = y_{i,j}$ and attains $\ell_{\text{sat}} = \frac{1}{\phi} \sum (y \log y - y)$. Twice the gap, times ϕ , is (7):

$$D = 2\phi(\ell_{\text{sat}} - \ell(\mu)). \quad (9)$$

Since ϕ only multiplies the whole log-likelihood, maximising (8) and minimising (7) give the same $\hat{\mu}$ for every ϕ : the dispersion plays no part in the point estimates (Härkönen, p. 5, makes the same remark). This is why the network can use the deviance as its loss and still estimate ϕ separately.

3.4 The maximum likelihood equations

Write $a_{i,j} = \log \mu_{i,j} = c + \alpha_i + \beta_j$. Differentiating one summand of (7) in $a_{i,j}$,

$$\frac{\partial}{\partial a_{i,j}} 2(e^{a_{i,j}} - y_{i,j} + y_{i,j} \log y_{i,j} - y_{i,j} a_{i,j}) = 2(\mu_{i,j} - y_{i,j}). \quad (10)$$

By the chain rule, with $\partial a_{i,j} / \partial c = 1$, $\partial a_{i,j} / \partial \alpha_{i'} = \mathbf{1}\{i = i'\}$ and $\partial a_{i,j} / \partial \beta_{j'} = \mathbf{1}\{j = j'\}$,

$$\begin{aligned} \frac{\partial D}{\partial c} &= 2 \sum_{(i,j) \in \mathcal{O}} (\mu_{i,j} - y_{i,j}), \\ \frac{\partial D}{\partial \alpha_i} &= 2 \sum_{j \leq n+1-i} (\mu_{i,j} - y_{i,j}), & \frac{\partial D}{\partial \beta_j} &= 2 \sum_{i \leq n+1-j} (\mu_{i,j} - y_{i,j}). \end{aligned} \quad (11)$$

At the MLE all of these vanish (for $i, j \geq 2$ directly; for $i = 1$ and $j = 1$ because the total minus the others vanishes too). So the fitted ccODP reproduces every observed row total and every observed column total of the triangle:

$$\begin{aligned} \sum_{j \leq n+1-i} \hat{\mu}_{i,j}^{\text{cc}} &= \sum_{j \leq n+1-i} y_{i,j} & (i = 1, \dots, n), \\ \sum_{i \leq n+1-j} \hat{\mu}_{i,j}^{\text{cc}} &= \sum_{i \leq n+1-j} y_{i,j} & (j = 1, \dots, n). \end{aligned} \quad (12)$$

This “marginal totals” property is the whole reason the ccODP is the chain ladder (Section 3.6), and it is also why two of the network’s gradients vanish at the start (Section 6.4). The numerical check finds the row and column sums of $\hat{\mu} - y$ at 10^{-11} .

3.5 How glm() solves them: IWLS

`stats::glm()` solves (11) by Fisher scoring, which for a GLM is iteratively weighted least squares. With the design matrix X (210×39 : a column of ones, 19 accident-period dummies, 19 development-period dummies) and coefficient vector $\vartheta = (c, \alpha_2, \dots, \alpha_n, \beta_2, \dots, \beta_n)$, each iteration computes, from the current ϑ ,

$$\begin{aligned} \eta &= X\vartheta, & \mu &= e^\eta, & z &= \eta + \frac{y - \mu}{\mu} \text{ (working response),} \\ W &= \text{diag}(\mu) \text{ (working weights),} & \vartheta_{\text{new}} &= (X^\top W X)^{-1} X^\top W z. \end{aligned} \quad (13)$$

For the log link the working weight μ comes from $(\partial \mu / \partial \eta)^2 / \text{Var} = \mu^2 / \mu$. The code runs it with `glm.control(epsilon = 1e-12, maxit = 100)`, so the relative change of the deviance between iterations has to fall below 10^{-12} ; it warns if that is not reached and stops if a coefficient is `NA` (the fitted cells then

do not connect all periods). The same routine fits a subset of the observed cells when a `cells` mask is given (rolling-origin partitions); then (11)-(12) hold over the fitted cells only.

3.6 The ccODP is the chain ladder

Statement. Let $\hat{\mu}^{\text{cc}}$ be the ccODP MLE on the full observed triangle, and define the fitted and predicted cumulative payments $\hat{C}_{i,j} = \sum_{l \leq j} \hat{\mu}_{i,l}^{\text{cc}}$. Then $\hat{C}_{i,n}$ equals the chain-ladder ultimate of accident period i , the latest cumulative $C_{i,n+1-i} = \sum_{j \leq n+1-i} y_{i,j}$ multiplied by the volume-weighted chain-ladder factors,

$$\hat{C}_{i,n} = C_{i,n+1-i} \prod_{k=n+1-i}^{n-1} f_k, \quad f_k = \frac{\sum_{i' \leq n-k} C_{i',k+1}}{\sum_{i' \leq n-k} C_{i',k}}, \quad (14)$$

so that the ccODP reserve $\hat{R} = \sum_{\mathcal{F}} \hat{\mu}_{i,j}^{\text{cc}}$ is the chain-ladder reserve. (Hachemeister and Stanard 1975; Mack 1991; Renshaw and Verrall 1998, the reference Härkönen gives as [13]; Paper C, Section 2.) The script asserts it: `all.equal(res$odp$reserve * scale, cl$total[["ibnr"]])`.

Proof. Write $\hat{\mu}_{i,j}^{\text{cc}} = \hat{a}_i \hat{b}_j$ with $\hat{a}_i = e^{\hat{c} + \hat{\alpha}_i}$ and $\hat{b}_j = e^{\hat{\beta}_j}$, and $S_k = \sum_{j \leq k} \hat{b}_j$, so that $\hat{C}_{i,k} = \hat{a}_i S_k$. Fix $k \in \{2, \dots, n\}$.

Step 1. Sum the column equations of (12) over columns $1, \dots, k$. The cells involved are the observed cells in those columns. They split into the rows $i \leq n + 1 - k$, for which all cells $(i, 1), \dots, (i, k)$ are observed, and the rows $i \geq n + 2 - k$, whose observed cells all lie in columns $\leq n + 1 - i \leq k - 1$, so the whole observed row is included. For those later rows the row equations of (12) say fitted row total = observed row total; subtracting them leaves

$$\sum_{i \leq n+1-k} \hat{C}_{i,k} = \sum_{i \leq n+1-k} C_{i,k}. \quad (15)$$

Step 2. Apply (15) with $k - 1$ in place of k : it runs over the rows $i \leq n + 2 - k$. The extra row $i = n + 2 - k$ has observed cells exactly up to column $k - 1$, so its row equation reads $\hat{C}_{n+2-k,k-1} = C_{n+2-k,k-1}$. Subtracting it,

$$\sum_{i \leq n+1-k} \hat{C}_{i,k-1} = \sum_{i \leq n+1-k} C_{i,k-1}. \quad (16)$$

Step 3. Divide (15) by (16): the right-hand side is the chain-ladder factor f_{k-1} of (14); the left-hand side is $\hat{a}_\bullet S_k / (\hat{a}_\bullet S_{k-1})$ with $\hat{a}_\bullet = \sum_{i \leq n+1-k} \hat{a}_i$. Hence

$$\frac{S_k}{S_{k-1}} = f_{k-1}, \quad k = 2, \dots, n. \quad (17)$$

Step 4. The row equation of (12) for accident period i says $\hat{a}_i S_{n+1-i} = C_{i,n+1-i}$. Therefore $\hat{C}_{i,n} = \hat{a}_i S_n = C_{i,n+1-i} S_n / S_{n+1-i} = C_{i,n+1-i} \prod_{k=n+2-i}^n S_k / S_{k-1}$, which by (17) is (14). $\square$

Two consequences are used later. First, (17) says the fitted development pattern of the ccODP is the chain-ladder pattern, the same for every accident period (Section 9). Second, the proof uses only the marginal-total equations (12); the bCCNN does not satisfy them after the first gradient step, so its reserve is no longer a chain-ladder reserve.

3.7 Dispersion

The point estimates do not need ϕ , but the variance (4), the Pearson residuals and any standard error do. `fit_odp_glm()` returns both usual estimators, with the degrees of freedom $n_{\text{obs}} - n_{\text{par}} = 210 - 39 = 171$ on the full triangle:

$$\begin{aligned}\hat{\phi}_D &= \frac{D(\mathbf{y}, \hat{\boldsymbol{\mu}}^{\text{cc}}; \mathcal{O})}{n_{\text{obs}} - n_{\text{par}}} \quad (\text{Paper C eq. (5), the default}), \\ \hat{\phi}_P &= \frac{1}{n_{\text{obs}} - n_{\text{par}}} \sum_{\mathcal{O}} \frac{(y_{i,j} - \hat{\mu}_{i,j}^{\text{cc}})^2}{\hat{\mu}_{i,j}^{\text{cc}}}.\end{aligned}\tag{18}$$

Both are moment estimators of the ratio variance-to-mean: under (4) each Pearson term has expectation about ϕ , and the deviance term agrees with it to second order. `config.yml` selects `odp: phi: deviance`. The dispersion of the bCCNN is a separate matter (Section 8).

3.8 The unit of the payments

The fit runs on $\mathbf{y}/u$ with $u = 10^6$. Let $\boldsymbol{\mu}'$ be the MLE on $\mathbf{y}' = \mathbf{y}/u$. Since $\log(\mu_{i,j}/u) = (c - \log u) + \alpha_i + \beta_j$, the scaled model is the same model with intercept shifted, and the deviance scales linearly:

$$\begin{aligned}D(\mathbf{y}/u, \boldsymbol{\mu}/u) &= \frac{1}{u} D(\mathbf{y}, \boldsymbol{\mu}), \quad \text{so} \\ \hat{c}' &= \hat{c} - \log u, \quad \hat{\alpha}' = \hat{\alpha}, \quad \hat{\beta}' = \hat{\beta}, \quad \hat{\boldsymbol{\mu}}' = \hat{\boldsymbol{\mu}}/u, \quad \hat{R}' = \hat{R}/u, \quad \hat{\phi}' = \hat{\phi}/u.\end{aligned}\tag{19}$$

The Pearson residual $(y - \mu)/\sqrt{\phi\mu}$ is invariant (u cancels), and so is the relative difference $\mu^{\text{bCCNN}}/\mu^{\text{cc}} - 1$. The numerical check confirms (19) to 10^{-13} . For the network the unit matters only through the fixed inputs: α_i and β_j do not change with u , so the hidden layers see the same inputs whatever the unit; the output intercept starts at $\hat{c}'$. Unlike SGD, RMSprop's steps are (almost) invariant to the scale of the gradient (companion document, Section 5), so the choice of unit changes the training very little.

3.9 Periods without payments

What happens. If an accident period or a development period has no payments in the fitted cells, its MLE is $-\infty$. In the row or column equation (12) every fitted mean of that period must sum to zero, which only $e^{-\infty} = 0$ achieves, so the deviance keeps decreasing as the effect goes to $-\infty$. `glm()` stops at a large negative value (about -30) that it reports as converged, with fitted means of order e^{-30} . `fit_odp_glm()` flags these periods as `zero_origin` and `zero_dev` and warns.

For the chain ladder this is harmless. A development period with no payments has chain-ladder factor exactly 1, and the ccODP limit $\mu = 0$ for that column predicts the same. The difficulty is purely numerical, and it matters in two places:

- **As a network input.** The embedding of that period is then ≈ -30 , far outside the range of the other effects (companion II, Section 6, on the raw inputs). The hidden layers see a value they were never trained near.
- **In the deviance.** A cell with $y > 0$ scored against $\mu \approx e^{-30}$ adds $y \log(y/\mu) \approx 30y$ to the deviance. One such cell can dominate a validation or test loss: in the check of the code change, a single

validation cell raised the starting validation deviance of a partition from 32 to 3,047. `poisson_deviance()` does not catch it, because e^{-30} is positive and finite.

What Paper C and Härkönen do. Neither discusses it. Härkönen replaces negative aggregated payments with zero so that the Poisson loss applies (her Sections 3.1 and 4.1), which makes every cell valid but does nothing about a period summing to zero. When she removes the last diagonal she notes that the networks “require input values for each accident year, reporting delay and payment delay” and declares them “not applicable” (Section 3.2), instead of handling the gap. Her triangles have 12 accident years and portfolios of about 100,000 to over a million claims, and Paper C’s six simulated lines of business are of the same kind, so every period has payments in both halves of their balanced claims splits. Paper C’s Listing 4 sets the embeddings to the GLM estimates with no guard. (Paper C is cited here through the quotations in your hyperparameter notes.)

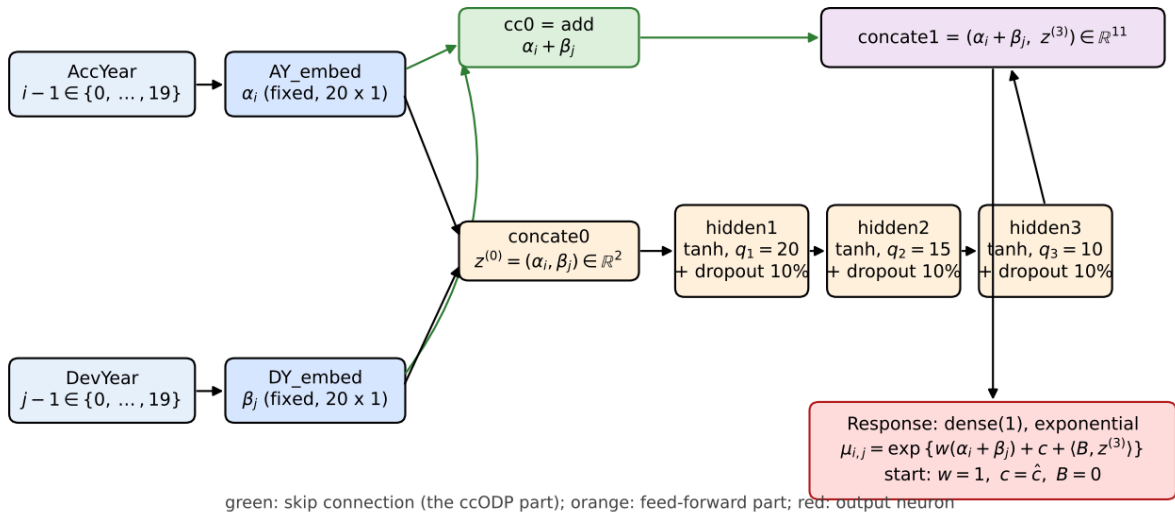
What the code does. The helper `mask_zero_periods(m, fit)` in `R/nn_models.R` sets to `NA` the cells of `m` in the periods `fit` flags, so they are left out of the deviance:

- under the claims split, `bccnn_validation()` masks the validation cells in periods with no payments in the training half (companion II, Section 3.3);
- under the rolling origin, `bccnn_ro_partition()` masks the validation cells in periods with no payments in the partition’s training cells, and the test cells in periods with no payments in *any* cell observed at the valuation date (companion III, Section 7). `n_vali` and `n_test` count the cells actually scored.

The network input itself is left as it is. On the full annual triangle every period has payments; whether a training half or a rolling-origin partition has an empty period depends on how sparse the last development years are.

4 The bCCNN network

Paper C, Section 3, Listing 4; Härkönen, Section 2.3.3. The network is drawn in Figure 1 with the layer names of `bccnn_model()`.



The network of `bccnn_model()` with Keras's layer names. The green path is the skip connection, which carries the ccODP linear predictor unchanged to the output; the orange path is the feed-forward network; the output neuron adds the two on the log scale and exponentiates.

4.1 Embedding layers

The inputs are the categorical accident period and development period of a cell, passed to Keras as the 0-based integers $i - 1$ and $j - 1$ (`bccnn_inputs()`). An embedding layer maps a category to a vector; here each is one-dimensional and holds the ccODP effect of that period (Paper C's eqs. (9)-(10), Härkönen's (33)-(34)):

$$\text{AY_embed} : i \mapsto \hat{\alpha}_i, \quad \text{DY_embed} : j \mapsto \hat{\beta}_j. \quad (20)$$

The two embedding matrices (20×1 each, 40 numbers) are set to the MLE with `set_weights()` and frozen (`trainable = trainable_embeddings, false` in `config.yml`), as Härkönen does and as Paper C's Listing 4 does. If they were trainable, 40 parameters would be added to θ , the skip connection would stop being exactly the ccODP after the first step, and Härkönen reports that the predictions improved for some lines of business at a higher computation cost (her "TW" rows). Everything below assumes they are fixed.

4.2 The skip connection

`cc0 = layer_add(AY_flat, DY_flat)` is the ccODP linear predictor without its intercept,

$$\text{cc0}(i, j) = \hat{\alpha}_i + \hat{\beta}_j. \quad (21)$$

It goes straight to the output neuron (the green path of Figure 1). This is Härkönen's "skip-connection" and Paper C's "embedding" of the ccODP: the classical model is a sub-network of the bCCNN.

4.3 The feed-forward part

The same two numbers are also concatenated into the input of a feed-forward network,

$$z^{(0)}(i, j) = (\hat{\alpha}_i, \hat{\beta}_j)^\top \in \mathbb{R}^2, \quad (22)$$

followed by three dense layers with hyperbolic tangent activation and $q_1 = 20$, $q_2 = 15$, $q_3 = 10$ neurons (Härkönen eq. (40); Paper C Listing 4):

$$z^{(h)}(i, j) = \tanh\left(b^{(h)} + W^{(h)\top} z^{(h-1)}(i, j)\right) \in \mathbb{R}^{q_h}, \quad h = 1, 2, 3, \quad (23)$$

where $\tanh$ acts component-wise, $W^{(h)} \in \mathbb{R}^{q_{h-1} \times q_h}$ and $b^{(h)} \in \mathbb{R}^{q_h}$. A dropout layer with rate $p = 0.1$ follows each hidden layer (Section 4.5). Härkönen gives the two reasons for $\tanh$: its derivative is $1 - \tanh^2$, cheap to compute from the activation itself (Section 6.3), and its range $(-1, 1)$ keeps the activations bounded.

The trainable parameters are counted in Table 1; the total 547 agrees with Keras’s `summary()` of the model (the 40 embedding numbers appear there as non-trainable).

layer	input $\times$ output	weights	biases	trainable parameters
hidden1	2×20	40	20	60
hidden2	20×15	300	15	315
hidden3	15×10	150	10	160
Response	11×1	11	1	12
total trainable				547
embeddings (frozen)	20×1 , twice	40	0	0 (40 non-trainable)

Table 1: Parameters of the network, Härkönen’s eq. (76) with $q_0 = 2$, $q_4 = 1$, plus the skip weight.

4.4 The output neuron and eq. (13)

`concat1` joins the skip connection and the last hidden layer into an 11-vector $(\text{cc}_0(i, j), z^{(3)}(i, j)) \in \mathbb{R}^{11}$, and `Response` is a dense layer with one neuron and exponential activation. Writing its 11 weights as (w, B) with $w \in \mathbb{R}$ on the skip connection and $B \in \mathbb{R}^{10}$ on the hidden features, and its bias as c , the network’s mean for cell (i, j) is Paper C’s eq. (13):

$$\mu_{i,j}(\theta) = \exp \left\{ \underbrace{w(\hat{\alpha}_i + \hat{\beta}_j)}_{\text{skip connection}} + \underbrace{c + \langle B, z^{(3)}(i, j) \rangle}_{\text{network}} \right\}, \quad a_{i,j} := \log \mu_{i,j}. \quad (24)$$

Paper C keeps w trainable, as `bccnn_model()` does (your comment “exactly as Listing 4 does”); Härkönen’s eq. (44) writes the skip connection with weight 1. Both views of (24) are useful:

- **Multiplicative boosting.** Factor out the ccODP mean: $\mu_{i,j} = \hat{\mu}_{i,j}^{\text{cc}} \cdot \exp\{(w - 1)(\hat{\alpha}_i + \hat{\beta}_j) + (c - \hat{c}) + \langle B, z^{(3)}(i, j) \rangle\}$. The network learns a multiplicative correction of the chain-ladder means, which is 1 at the start. This is the CANN (combined actuarial neural network) idea of Wüthrich and Merz (2019) that Härkönen cites as [15], and Paper C’s “neural network boosting”.
- **A function of two scalars.** The feed-forward part sees only $(\hat{\alpha}_i, \hat{\beta}_j)$. Whatever it learns is a smooth surface $g(\alpha, \beta)$ evaluated at the 20 accident effects and 20 development effects. Two accident periods with equal $\hat{\alpha}$ receive the same correction at every development period; the correction for a future cell (i, j) is the surface at the pair $(\hat{\alpha}_i, \hat{\beta}_j)$, a combination never seen in training even though each coordinate was. What the network can add to the ccODP is therefore exactly an interaction between the

accident-period effect and the development-period effect, that is, accident-period-dependent development patterns (Section 9, cumulative factors).

4.5 Dropout

Keras’s dropout is “inverted dropout” (Srivastava et al. 2014, Härkönen’s [14] and eq. (31)-(32)). During training, after each hidden layer a mask of independent Bernoulli variables is drawn and the surviving activations are scaled up so that their expectation is unchanged:

$$\begin{aligned} \tilde{z}^{(h)} &= \frac{z^{(h)} \odot m^{(h)}}{1 - p}, & m_r^{(h)} &\sim \text{Bernoulli}(1 - p) \text{ independently,} \\ \mathbb{E}[\tilde{z}^{(h)} \mid z^{(h)}] &= z^{(h)}. \end{aligned} \quad (25)$$

The next layer reads $\tilde{z}^{(h)}$ in place of $z^{(h)}$ in (23), and the output neuron reads $\tilde{z}^{(3)}$ in (24). At prediction time (`predict_on_batch`, `predict`) no mask is drawn and (23)-(24) hold as written. Two consequences for what the code records (Section 5.2): the loss Keras prints during training is the loss of a randomly thinned network, and it is reported as the column `train_dropout`; the deviances in the columns `train`, `vali`, `test`, `truth` are computed from `predict_on_batch()` after each step, without dropout. Paper C’s Figure 2 shows the first kind of curve (noisy), Table 3 reports both kinds. Härkönen’s remark on dropout, that each neuron must learn features that are useful without relying on particular other neurons, is the reason it regularises a network with 547 parameters on 210 cells.

4.6 The initialisation: Paper C’s eq. (14)

Keras builds the model with its default initialisers: Glorot-uniform weights, $W_{r,s}^{(h)} \sim \text{U}(-\lambda_h, \lambda_h)$ with $\lambda_h = \sqrt{6/(q_{h-1} + q_h)}$, and zero biases, for the three hidden layers and for `Response`; `set_random_seed(seed)` fixes the draw. Then `bccnn_model()` overwrites the embeddings and the output neuron:

$$\begin{aligned} \theta_0 : \quad & \text{AY_embed} = \hat{\alpha}, & \text{DY_embed} &= \hat{\beta}, \\ & w = 1, & c &= \hat{c}, & B &= 0 \in \mathbb{R}^{10}, \\ & (W^{(h)}, b^{(h)}) & \text{random: Glorot-uniform weights, zero biases,} & & h &= 1, 2, 3. \end{aligned} \quad (26)$$

Property 1: the start is the ccODP model. With $B = 0$ the hidden layers do not reach the output, whatever their random weights, and (24) becomes $\mu_{i,j}(\theta_0) = \exp\{\hat{\alpha}_i + \hat{\beta}_j + \hat{c}\} = \hat{\mu}_{i,j}^{\text{cc}}$ for every cell, observed or future. Hence the deviance at step 0 equals the ccODP deviance, the reserve at step 0 equals the chain-ladder reserve (Section 3.6), and every cumulative development factor at step 0 is the chain-ladder factor. The record at `epoch = 0` of the history is this start. Keras holds the weights in single precision, so in R the equality holds to about 10^{-7} relative; in the double-precision check it holds to 10^{-14} .

This is what makes the bCCNN a calibration of the chain ladder rather than a new model: the number of gradient steps measures how far the network is allowed to move away from the chain ladder, and $t^* = 0$ returns the chain ladder itself.

INFERRED, your comment in `bccnn_fit.R`. The Python version of the single bCCNN used torch’s default initialiser, $\text{U}(\pm 1/\sqrt{\text{fan-in}})$, for the hidden layers, with the observation that it “leaves the chain ladder about three times more slowly”. The reason is visible in Section 6.4: the only non-zero gradient at the start is the one in B , and it is proportional to the hidden activations $z^{(3)}$; smaller initial weights make $z^{(3)}$

smaller and closer to zero, so the first steps add less. Paper C’s own default is Keras’s, which the R code keeps.

5 The loss as Keras sees it

5.1 Keras’s “poisson” loss and the deviance

`compile(loss = "poisson")` minimises the mean over the N training cells of $\mu - y \log(\mu + \varepsilon)$, with Keras’s small $\varepsilon = 10^{-7}$ inside the logarithm,

$$\ell(\theta) = \frac{1}{N} \sum_{r=1}^N \left(\mu_r(\theta) - y_r \log(\mu_r(\theta) + \varepsilon) \right). \quad (27)$$

Dropping ε (its effect on ℓ is below 10^{-7} here), (27) and the deviance (7) over the same cells differ by a constant that does not depend on θ :

$$D(\mathbf{y}, \boldsymbol{\mu}(\theta); \mathcal{S}) = 2N \ell(\theta) + 2 \sum_{r=1}^N (y_r \log y_r - y_r). \quad (28)$$

So minimising Keras’s loss is minimising the Poisson deviance, the same criterion the ccODP was fitted by (Paper C, Section 3.3; Härkönen, Section 2.3.5, eq. (72)), and the gradients differ only by the factor $2N$. The function `keras_poisson_to_deviance(loss, y)` is (28); the code applies it to the loss Keras reports after each epoch to obtain the column `train_dropout` in the units of the other columns.

5.2 What is recorded after every step

`fit_bccnn()` keeps every cell of the $n \times n$ square in `x_all` and the training cells in `x_fit`. After each gradient step a Keras callback runs `predict_on_batch(x_all)` (no dropout), reshapes the result to the $n \times n$ matrix $\boldsymbol{\mu}^{(t)}$ and stores one row of the history:

column	formula	cells
<code>epoch</code>	step t (0 = the ccODP start)	
<code>train_dropout</code>	$2N \ell_t^{\text{drop}} + 2 \sum (y \log y - y)$, from the loss Keras reported for epoch t (with dropout; computed before that epoch’s update, so it belongs to $\boldsymbol{\theta}_{t-1}$)	training cells
<code>train</code>	$D(\mathbf{y}, \boldsymbol{\mu}^{(t)}; \text{training cells})$	training cells
<code>vali, test, truth</code>	$D(\cdot, \boldsymbol{\mu}^{(t)}; \cdot)$ on the cells of each tracked matrix (NA elsewhere)	validation, test, true lower triangle
<code>*_pred</code>	$\sum \mu_{i,j}^{(t)}$ over the cells of each tracked matrix	the same

Table 2: The history of `fit_bccnn()`.

With `batch_size = NULL` the batch is all N training cells, so one Keras epoch is exactly one gradient-descent step and `epoch` counts steps, as in Paper C (“gradient descent iteration”). The companion document uses these columns for the stopping rule.

6 Gradients

The gradient Keras computes by automatic differentiation is written out here for the loss (27); for the deviance multiply by $2N$. Everything is per cell $r = (i, j)$ of the training set, summed over the batch.

6.1 Derivative in the output

From (10) and (27), with $a_r = \log \mu_r$,

$$\delta_r := \frac{\partial \ell}{\partial a_r} = \frac{1}{N} (\mu_r - y_r). \quad (29)$$

The signal that drives every parameter is the residual $\mu_r - y_r$ of the cell: positive where the network over-predicts, negative where it under-predicts, and zero in a cell that is fitted exactly.

6.2 Output layer

From (24), $\partial a_r / \partial c = 1$, $\partial a_r / \partial w = \hat{\alpha}_i + \hat{\beta}_j$ and $\partial a_r / \partial B = \tilde{z}_r^{(3)}$, so

$$\frac{\partial \ell}{\partial c} = \sum_r \delta_r, \quad \frac{\partial \ell}{\partial w} = \sum_r \delta_r (\hat{\alpha}_i + \hat{\beta}_j), \quad \frac{\partial \ell}{\partial B} = \sum_r \delta_r \tilde{z}_r^{(3)} \in \mathbb{R}^{10}. \quad (30)$$

6.3 Hidden layers (back-propagation)

Let $\gamma_r^{(h)} \in \mathbb{R}^{q_h}$ be the derivative of ℓ in the pre-activation $u_r^{(h)} = b^{(h)} + W^{(h)\top} \tilde{z}_r^{(h-1)}$ of layer h . Since $z = \tanh(u)$ has $\partial z / \partial u = 1 - z^2$, and the mask of (25) multiplies the activation,

$$\begin{aligned} \gamma_r^{(3)} &= \delta_r \left(B \odot \frac{m_r^{(3)}}{1-p} \right) \odot (1 - z_r^{(3)2}), \\ \gamma_r^{(h)} &= \left(W^{(h+1)} \gamma_r^{(h+1)} \odot \frac{m_r^{(h)}}{1-p} \right) \odot (1 - z_r^{(h)2}), \quad h = 2, 1, \end{aligned} \quad (31)$$

and the parameter gradients are

$$\begin{aligned} \frac{\partial \ell}{\partial W^{(h)}} &= \sum_r \tilde{z}_r^{(h-1)} \gamma_r^{(h)\top} \in \mathbb{R}^{q_{h-1} \times q_h}, \quad \frac{\partial \ell}{\partial b^{(h)}} = \sum_r \gamma_r^{(h)}, \\ h &= 1, 2, 3, \quad \tilde{z}_r^{(0)} = z_r^{(0)}. \end{aligned} \quad (32)$$

Without dropout set every mask to $1 - p$ so the factors $m/(1-p)$ become 1. The numerical check compares (30)-(32) (no dropout) with central finite differences of the deviance at the start and at a random point: the largest relative error over all nine parameter groups is 1.1×10^{-6} (in $W^{(3)}$), the others are at 10^{-7} or below (Section 11).

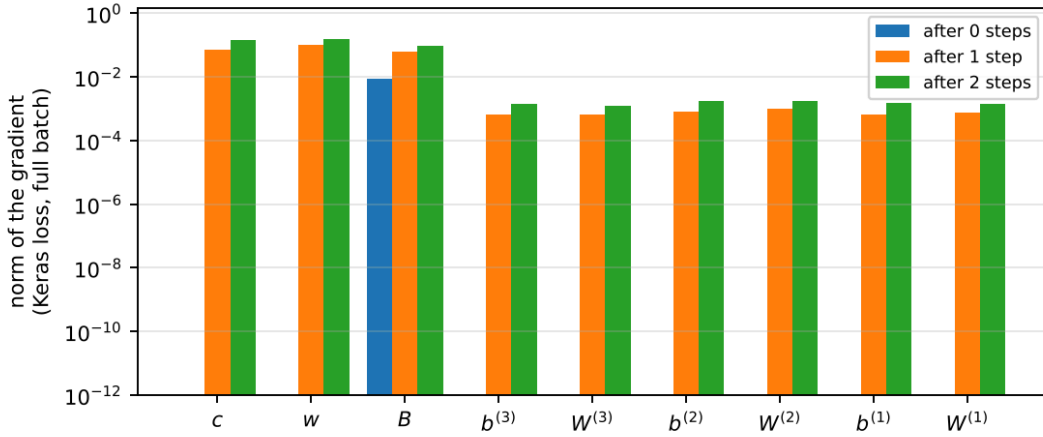
6.4 Property 2: the gradient at the start

At θ_0 the means are the ccODP means, so $\delta_r = (\hat{\mu}_r^{\text{cc}} - y_r)/N$, and the marginal-total equations (12) give:

- $\partial \ell / \partial c = \frac{1}{N} \sum_{\mathcal{O}} (\hat{\mu}^{\text{cc}} - y) = 0$ (the total);

- $\partial\ell/\partial w = \frac{1}{N}(\sum_i \hat{\alpha}_i \sum_j (\hat{\mu}_{i,j}^{\text{cc}} - y_{i,j}) + \sum_j \hat{\beta}_j \sum_i (\hat{\mu}_{i,j}^{\text{cc}} - y_{i,j})) = 0$ (row and column totals);
- every hidden-layer gradient is zero, because $B = 0$ makes $\gamma_r^{(3)} = 0$ in (31) and hence $\gamma^{(2)} = \gamma^{(1)} = 0$;
- $\partial\ell/\partial B = \frac{1}{N} \sum_r (\hat{\mu}_r^{\text{cc}} - y_r) \tilde{z}_r^{(3)}$ is the only non-zero component.

So the first gradient step changes nothing but B : it adds to the ccODP log-mean the linear combination $\langle \Delta B, z^{(3)}(i, j) \rangle$ of the ten random hidden features that is most correlated (in the sense of (30)) with the chain-ladder residuals. This is a boosting step on the residuals of the chain ladder with random features, nothing else. From the second step on, $B \neq 0$ and the gradient flows back into the hidden layers, which then start to shape the features themselves; c and w also start to move, because the marginal-total equations no longer hold once $B \neq 0$. Figure 2 shows the norms of the nine gradient groups over the first steps in the numerical check: at step 0 only B is non-zero (c and w are at 10^{-14} , the hidden layers at exactly 0); after one step all are non-zero, with c , w and B two orders of magnitude above the hidden layers.



Norms of the gradient of Keras's loss by parameter group at the start and after one and two full-batch RMSprop steps, in the numerical check (no dropout). Bars missing at step 0 are exactly zero or at machine precision.

This property holds on the full triangle and on the training half of the claims split (the ccODP is refitted on the training cells, so (12) holds there). It holds equally for a rolling-origin partition, where the ccODP is fitted on the training cells of the partition by the same `glm()` call: (12) then holds over those cells.

6.5 Property 3: the size of the first steps

The optimiser is RMSprop with Keras's parameterisation (companion document, Section 5): for each coordinate ϑ of θ with gradient g_t ,

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2, \quad \vartheta_t = \vartheta_{t-1} - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}, \quad v_0 = 0. \quad (33)$$

At the first step $v_1 = (1 - \rho) g_1^2$, so the update is $\eta g_1 / (\sqrt{1 - \rho} |g_1| + \epsilon)$. For every coordinate whose gradient is not tiny ($|g_1| \gg \epsilon / \sqrt{1 - \rho} \approx 3 \times 10^{-7}$) this is

$$\Delta \vartheta_1 \approx -\frac{\eta}{\sqrt{1 - \rho}} \text{sign}(g_1) = \mp 0.00316, \quad (34)$$

independent of the size of the gradient. Hence the first step moves each of the ten coordinates of B by ± 0.00316 exactly, and nothing else (the coordinates c and w , whose gradient is 10^{-14} , move by $\eta g/\epsilon \approx 10^{-10}$). The check reports $\max |\Delta B_1| = 0.0031621$ against 0.0031623 from (34). Two things follow. First, the early path away from the chain ladder is set by η , ρ and the random features, not by how badly the chain ladder fits: a triangle the chain ladder fits well and one it fits badly both move by the same amount at step 1. Second, because the step is sign-like rather than proportional, the training deviance need not fall at first: in the check it goes $108.967 \rightarrow 109.034 \rightarrow 109.198 \rightarrow 108.973$ over the first three steps before it starts to decrease. Paper C’s and your loss curves start with the same kind of wobble.

7 From the fitted means to the reserves

Whatever the model, the reserve is the sum of the predicted means over the future cells, Paper C’s Section 3.3.3:

$$\hat{R}_i = \sum_{j>n+1-i} \mu_{i,j}, \quad \hat{R} = \sum_{i=1}^n \hat{R}_i = \sum_{(i,j) \in \mathcal{F}} \mu_{i,j}. \quad (35)$$

`reserve_by_origin(y, mu)` returns, per accident period, the latest cumulative $C_{i,n+1-i}$ (latest), $\hat{R}_i$ (ibnr), their sum (ultimate) and the ratio `dev_to_date` $= C_{i,n+1-i}/(C_{i,n+1-i} + \hat{R}_i)$; `reserve_totals()` sums them. For the ccODP, (35) is the chain-ladder reserve by Section 3.6; for the bCCNN it is the sum of (24) over $\mathcal{F}$ at the chosen step. There are no standard errors in this strand of the code (`se`, `cv` are `NA`); the companion *The bCCNN as a whole* discusses how the uncertainty of (35) can be obtained.

8 Dispersion of the bCCNN

A bCCNN has 547 parameters and is trained on 210 cells, so the GLM estimator (18) with $n_{\text{obs}} - n_{\text{par}}$ in the denominator does not exist for it; Härkönen notes exactly this for her networks (Section 5: “the number of parameters [...] is even larger than the number of observations which implies that the estimated overdispersion parameter cannot be computed according to Pearson statistics”). Paper C, Section 3.3.2 and Table 2, uses a heuristic instead: the ccODP dispersion is reduced by the relative decrease of the *validation* loss that the network achieved,

$$\hat{\phi}^{\text{bCCNN}} = \hat{\phi}_D (1 - \max(0, \Delta_V)), \quad \Delta_V = 1 - \frac{D_V(t^*)}{D_V(0)}, \quad (36)$$

where $D_V(t)$ is the validation deviance after t steps (companion document). The logic is that the deviance is an estimate of ϕ times the degrees of freedom; if the network explains a fraction Δ_V of the out-of-sample deviance, the unexplained variance is reduced by that fraction. The cut at zero means a network that does not beat the chain ladder on the validation cells keeps the chain ladder’s dispersion (Paper C sets 0% for its LoBs 3 and 6). It is `bccnn_phi(phi_odp, decrease_vali)` and feeds the Pearson residuals of the bCCNN.

9 Diagnostics computed from the means

Pearson residuals (Paper C Figure 7; `pearson_residuals()`), on the true lower triangle in the script:

$$r_{i,j} = \frac{y_{i,j} - \mu_{i,j}}{\sqrt{\hat{\phi} \mu_{i,j}}}, \quad (37)$$

with the model's own $\hat{\phi}$ ((18) for the ccODP, (36) for the bCCNN). Under (4) they have mean 0 and variance 1 in the fitted cells; on the lower triangle they are out-of-sample standardised errors.

Cumulative development factors (Paper C Figure 8; `cum_dev_factors()`). From a mean square μ , the individual factor of accident period i from development period $j - 1$ to j and its cumulative version are

$$f(i, j) = \frac{\sum_{l \leq j} \mu_{i,l}}{\sum_{l \leq j-1} \mu_{i,l}}, \quad g(i, j) = \prod_{l=2}^j f(i, l) = \frac{\sum_{l \leq j} \mu_{i,l}}{\mu_{i,1}}, \quad j = 2, \dots, n. \quad (38)$$

Under the ccODP, $\mu_{i,l} = \hat{\alpha}_i \hat{b}_l$ gives $g(i, j) = S_j / \hat{b}_1 = S_j$ (since $\hat{\beta}_1 = 0$), the same for every accident period, and by (17) it equals the product of the chain-ladder factors $\prod_{l=2}^j f_{l-1}$: every row of the ccODP factor matrix is the chain-ladder pattern. The numerical check finds the spread across rows at 3×10^{-15} . The bCCNN breaks the product form through the interaction term $\langle B, z^{(3)}(i, j) \rangle$, so its $g(i, j)$ vary with i : the plot of (38) against j with one line per accident period shows directly how far the network has let the development pattern depend on the accident period.

Relative difference (Paper C Figure 5; `plot_relative_difference()`),

$$\frac{\mu_{i,j}^{\text{bCCNN}}}{\hat{\mu}_{i,j}^{\text{cc}}} - 1 = \exp \left\{ (w-1)(\hat{\alpha}_i + \hat{\beta}_j) + (c - \hat{c}) + \langle B, z^{(3)}(i, j) \rangle \right\} - 1, \quad (39)$$

is the multiplicative correction of Section 4.4 minus one, shown as a heat map over the whole square. It is zero everywhere at step 0.

10 Where each formula is in the code

formula	where
(1)-(3) cells, observed set, reserve to predict	<code>triangle_sets()</code> , <code>fut_mask()</code> , <code>upper()</code> , <code>fut_cells()</code> in <code>R/triangles.R</code>
(4)-(6) ccODP model and identification	<code>fit_odp_glm(): glm(y ~ origin + dev,</code> <code>family = quasipoisson(link = "log"))</code> with factors; <code>alpha <- c(0, coef[origin2..n])</code> , <code>beta</code> <code><- c(0, coef[dev2..n])</code>
(7) Poisson deviance	<code>poisson_deviance()</code> in <code>R/loss functions.R</code>
(11)-(13) MLE by IWLS	inside <code>stats::glm()</code> , <code>glm.control(epsilon =</code> <code>1e-12, maxit = 100)</code>
(14) chain-ladder equivalence	asserted in <code>bccnn fit.R</code> : <code>all.equal(res\$odp\$reserve * scale,</code> <code>cl\$total[["ibnr"]])</code> , with <code>fit_chainladder()</code> in <code>R/classical_models.R</code>
Section 3.9, periods without payments	<code>fit_odp_glm(): zero_origin, zero_dev;</code> <code>mask_zero_periods()</code> in <code>R/nn_models.R</code>
(18) dispersion	<code>fit_odp_glm(): phi_deviance, phi_pearson,</code> <code>n_par <- 2L * n - 1L</code>
(19) unit of the payments	<code>fit_odp_glm(scale =)</code> , <code>dat_cfg\$scale</code> ; every output in those units
(20)-(21) embeddings and skip connection	<code>layer_embedding(input_dim = n, output_dim</code> <code>= 1, trainable = ...)</code> , <code>set_weights(list(as.matrix(odp\$alpha)))</code> , <code>layer_add(..., name = "cc0")</code>
(22)-(23) feed-forward part	<code>layer_concatenate(name = "concat0"),</code> <code>layer_dense(units = q[h], activation =</code> <code>"tanh"), layer_dropout(rate = dropout)</code>
(24) output, eq. (13)	<code>layer_concatenate(name = "concat1"),</code> <code>layer_dense(units = 1, activation =</code> <code>"exponential", name = "Response")</code>
(26) the start, eq. (14)	<code>set_weights(list(as.matrix(c(1, rep(0,</code> <code>q[3]))), array(odp\$c)))</code> on <code>Response</code> ; <code>clear_session()</code> , <code>set_random_seed(seed)</code>
(27)-(28) Keras loss and deviance	<code>compile(loss = "poisson");</code> <code>keras_poisson_to_deviance()</code>
Table 2 history	<code>fit_bccnn(): record(),</code> <code>callback_lambda(on_epoch_end = ...)</code> , <code>predict_on_batch(x_all)</code>
(29)-(32) gradients	Keras automatic differentiation; written out here only
(33) RMSprop	<code>optimizer_rmsprop(learning_rate, rho,</code> <code>epsilon)</code> with <code>config.yml bccnn: training:</code>
(35) reserves	<code>reserve_by_origin(), reserve_totals()</code> in <code>R/metrics.R</code> , <code>R/classical_models.R</code>
(36) bccnn dispersion	<code>bccnn_phi(); loss_decrease()</code> for Δ_V
(37)-(39) diagnostics	<code>pearson_residuals(), cum_dev_factors()</code> in <code>R/metrics.R</code> ; <code>plot_relative_difference()</code> in

formula	where
	R/plots.R

11 Numerical check of the formulas

The check script (`bccnn_check.py`, kept with these explainers) builds a 20×20 triangle from a known ccODP structure plus an interaction term the ccODP cannot represent, draws over-dispersed Poisson payments in millions, and then recomputes everything above in numpy. The data is synthetic; it is there to test the formulas, not to say anything about your portfolio. Table 3 lists the results.

check	result
IWLS (13): chain-ladder reserve vs ccODP reserve (14)	relative difference 5.1×10^{-15}
marginal totals (12): largest row sum and column sum of $\hat{\mu} - y$	7.8×10^{-12} , 6.7×10^{-12}
chain-ladder factors from the data vs S_k/S_{k-1} of the fitted means (17), first five	1.68757, 1.28717, 1.17237, 1.10129, 1.06176, both
unit (19): $\hat{c}' - \hat{c}$ vs $-\log u$, $u = 1000$	-6.9077552790 vs -6.9077552790 ; $\max \hat{\alpha}' - \hat{\alpha} = 7 \times 10^{-15}$, $\max \hat{\beta}' - \hat{\beta} = 7 \times 10^{-13}$
cumulative factors (38) of the ccODP: spread across accident periods	3.1×10^{-15}
dispersion (18): $\hat{\phi}_D, \hat{\phi}_P$ (true $\phi = 0.4$ in these units; the misspecified interaction inflates both)	0.637, 0.622
parameters (Table 1): trainable, non-trainable	547, 40
Property 1 (26): $\max_{i,j} \mu_{i,j}(\theta_0)/\hat{\mu}_{i,j}^{cc} - 1 $	1.4×10^{-14} ; reserve and deviance at step 0 equal the ccODP's to all printed digits
gradients (30)-(32) vs central finite differences at the start: B (largest entry 2.08)	absolute error 1.1×10^{-7} ; c and w : analytic 10^{-12} , finite difference 10^{-8} (both zero to the accuracy of the difference quotient); hidden layers: exactly 0, finite differences exactly 0
gradients at a random point ($B \neq 0$): largest relative error by group	$W^{(1)} 1.1 \times 10^{-7}$, $b^{(1)} 9 \times 10^{-10}$, $W^{(2)} 2.4 \times 10^{-8}$, $b^{(2)} 3.7 \times 10^{-9}$, $W^{(3)} 1.1 \times 10^{-6}$, $b^{(3)} 6.8 \times 10^{-8}$, $B 2.4 \times 10^{-8}$, $w 3 \times 10^{-11}$, $c 1.5 \times 10^{-10}$
Keras loss identity (28), $N = 210$: $2N\ell + 2 \sum (y \log y - y)$ vs direct deviance	108.966655469307 vs 108.966655469310; effect of Keras's ε on ℓ : 9.7×10^{-8}
Property 3 (34): largest first-step update, B vs $\eta/\sqrt{1-\rho}$; all other groups	0.0031621 vs 0.0031623; hidden layers 0, $w 1.1 \times 10^{-10}$, $c 1.8 \times 10^{-10}$
training deviance over the first three RMSprop steps (no dropout)	108.967, 109.034, 109.198, 108.973

Table 3: Results of the numerical check (all amounts in millions of the synthetic triangle).

References

Al-Mudafer, M. T., Avanzi, B., Taylor, G. and Wong, B. (2021). Stochastic loss reserving with mixture density neural networks. arXiv:2108.07924.

- Gabrielli, A., Richman, R. and Wüthrich, M. V. (2020). Neural network embedding of the over-dispersed Poisson reserving model. *Scandinavian Actuarial Journal* 2020(1):1-29. (“Paper C”.)
- Gabrielli, A. (2020). A neural network boosted double over-dispersed Poisson claims reserving model. *ASTIN Bulletin* 50(1):25-60.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press.
- Hachemeister, C. A. and Stanard, J. N. (1975). IBNR claims count estimation with static lag functions. *ASTIN Colloquium*, Portimão.
- Härkönen, V. (2021). On claims reserving with machine learning techniques. Master thesis 2021:4, *Mathematical Statistics*, Stockholm University.
- Mack, T. (1991). A simple parametric model for rating automobile insurance or estimating IBNR claims reserves. *ASTIN Bulletin* 21(1):93-109.
- McCullagh, P. and Nelder, J. A. (1989). *Generalized Linear Models*, 2nd ed. Chapman and Hall.
- Renshaw, A. E. and Verrall, R. J. (1998). A stochastic model underlying the chain-ladder technique. *British Actuarial Journal* 4(4):903-923.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* 15:1929-1958.
- Wüthrich, M. V. and Merz, M. (2019). Editorial: Yes, we CANN! *ASTIN Bulletin* 49(1):1-3.